

GIORNO 04

Condizioni: insegnare al programma a decidere e a dire di no

Il post-it sul monitor

L'ultima volta Giulia ha consegnato la cassa di NovaStore.

Prezzo, quantità, contanti, IVA, resto: sette prove, sette risultati giusti.

Stamattina, sul monitor, un post-it della responsabile:

"La cassa si è chiusa da sola davanti a un cliente.

La ragazza nuova ha scritto tre invece di 3.

È uscita una scritta rossa e poi più niente.

Abbiamo finito la giornata con la calcolatrice."

Il programma non è rotto: fa esattamente quello che gli è stato insegnato.

Non sa guardare un dato, decidere se va bene e comportarsi di conseguenza.

Cosa saprete fare stasera

Far scegliere al programma fra due o più strade, con l'indentazione al posto giusto

Mettere in fila una catena di soglie nell'ordine che funziona

Combinare condizioni con and, or e not prevedendo il risultato

Controllare un dato prima di usarlo, e rispondere con un messaggio comprensibile

Trovare l'errore in un programma che gira e sbaglia lo stesso

Riprendere la cassa del Giorno 03 e aggiungerle i controlli che le mancavano

in Python gli spazi a inizio riga cambiano il significato del programma.

1. RIPASSO: LO SCRIPT CHE ESPLODE

E quello che sbaglia in silenzio

1.1 Il programma di partenza: legge, converte, calcola

```
# Cassa NovaStore - versione del Giorno 03
```

```
ALIQUOTA_IVA = 0.22
```

```
def main():
```

```
    # TRABOCCHETTO: lettura e conversione sono annidate sulla stessa riga.
```

```
    #   input() restituisce sempre una stringa, e float() o int() la
```

```
    #   convertono subito, senza che nessuno abbia guardato il contenuto.
```

```
    prezzo = float(input("Prezzo unitario: "))
```

```
    quantita = int(input("Quantità: "))
```

```
    contanti = float(input("Contanti ricevuti: "))
```

```
    # Il calcolo non ha difetti: prende i numeri che trova e li usa.
```

```
    imponibile = prezzo * quantita
```

```
    totale = imponibile * (1 + ALIQUOTA_IVA)
```

```
    resto = contanti - totale
```

```
    print(f"Totale: {totale:.2f} euro")
```

```
    print(f"Resto: {resto:.2f} euro")
```

```
if __name__ == "__main__":
```

```
    main()
```

1.2 Il programma senza controlli: ValueError e traceback

```
Prezzo unitario: 12.90
Quantità: tre
Traceback (most recent call last):
  File "/Users/giulia/corso-python/giorno_04/cassa.py", line 21, in <module>
    main()
  File "/Users/giulia/corso-python/giorno_04/cassa.py", line 9, in main
    quantita = int(input("Quantità: "))
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
ValueError: invalid literal for int() with base 10: 'tre'
```

Si legge dal basso. La parte importante non è il messaggio: è cosa succede dopo. Niente. Nessun totale, nessuna indicazione alla cassiera, e la fila davanti al banco.

1.2 Traceback, eccezione, letterale: i nomi giusti

Sono le parole con cui questa roba si cerca in una documentazione o si racconta a un collega. Da oggi le usiamo.

traceback = l'elenco dei punti del programma rimasti aperti quando si è fermato

eccezione = la segnalazione che Python solleva davanti a una situazione che non sa gestire; se nessuno la raccoglie, il programma termina
ValueError, TypeError, NameError, IndentationError sono nomi di eccezioni

letterale (literal) = un valore scritto dentro il testo del programma:
3, 12.90, "tre". Il messaggio dice che 'tre' non è un letterale numerico valido in base dieci: non "non ho capito".

1.3 L'errore logico: nessuna eccezione, risultato sbagliato

```
ALIQUOTA_IVA = 0.22

prezzo = 12.90

# TRABOCCHETTO: int() accetta il segno meno senza fiatare, perché "-3" è
# un letterale numerico intero a tutti gli effetti. Il tipo è giusto e
# il valore è assurdo: nessun controllo di Python se ne occupa.

quantita = -3
contanti = 50.00

# Da qui in poi l'aritmetica è ineccepibile. Un totale negativo produce un
# resto maggiore dei contanti ricevuti, e il conto torna perfettamente.

imponibile = prezzo * quantita
totale = imponibile * (1 + ALIQUOTA_IVA)
resto = contanti - totale

print(f"Totale: {totale:.2f} euro")
print(f"Resto: {resto:.2f} euro")

# Totale: -47.21 euro
# Resto: 97.21 euro
```

Con contanti 20 e quantità 3 esce Resto: -27.21. Con quantità -3 la cassa dispone novantasette euro di resto per una vendita mai avvenuta. Nessuna riga rossa.

1.3 Due modi di sbagliare, due mestieri diversi

Cosa succede	Come si chiama	Chi ve lo dice
Il programma si ferma	errore di esecuzione	Python, col traceback
Il programma risponde ma	errore logico	nessuno

Il secondo è enormemente peggiore. Un programma che si ferma vi fa perdere dieci minuti; uno che sbaglia in silenzio vi fa perdere la fiducia di chi lo usa, e ve ne accorgete l'anno dopo, quando qualcuno rifà i conti a mano.

1.4 Ripasso: input() dà str, .strip() una copia

```
# Due spazi davanti e due dietro: a schermo "  SUD  " e "SUD" si somigliano
# molto, e per == sono due stringhe diverse.

testo = "  SUD  "

# I due metodi si concatenano: .strip() restituisce una stringa nuova,
# .upper() ne restituisce un'altra a partire da quella. `testo` non cambia.
zona = testo.strip().upper()

print(zona == "SUD")

# True

# Senza .strip() sarebbe False, e a schermo i due valori sembrano identici.
```

input() legge dallo standard input e restituisce sempre un valore di tipo str, anche se l'utente ha digitato solo cifre. Le stringhe sono immutabili: .strip() non modifica, restituisce. Da qui la forma testo = testo.strip().

2. IL BIVIO

Flusso di esecuzione, ramo, condizione

2.1 Decidere: righe che a volte si eseguono e a volte no

Fino a oggi i programmi erano una lista della spesa: si esegue tutto, dall'alto in basso.

Decidere significa una cosa sola: esistono righe che a volte vengono eseguite e a volte no.

Quattro parole, e descrivete a voce qualunque if di oggi

flusso di esecuzione: l'ordine in cui Python esegue le righe

ramo: ognuna delle strade che si aprono

blocco: il gruppo di righe rientrate attaccato a un ramo

condizione: la domanda che decide quale ramo prendere

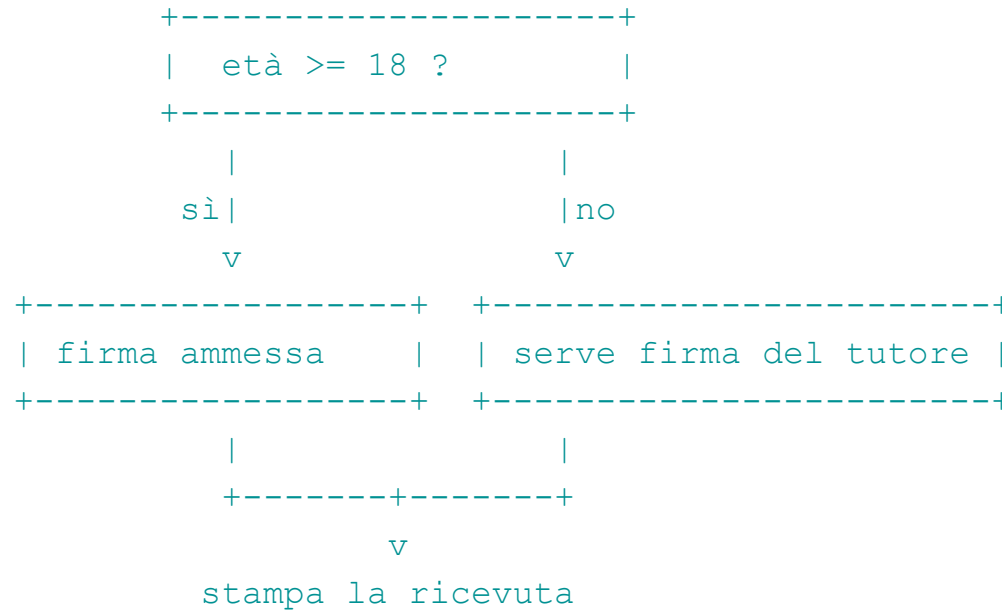
Analogia - il buttafuori all'ingresso

fa i controlli in ordine: prima il documento, poi l'età, poi la lista

se ti ferma il primo, gli altri non li fa nemmeno

la decisione è secca: entri o non entri, non esiste "entra un po"

2.2 Il diagramma di flusso: rombo e ricongiunzione



Il rombo è la decisione, il rettangolo è l'azione, la freccia è il flusso. Il rombo ha una domanda sola; i due rami sono alternativi; i rami si ricongiungono, e in Python quel punto non ha nessuna parola chiave: è la prima riga che torna al margine.

2.3 Le tre forme: if, if/else, if/elif/else

Forma	Quante strade	Quando si usa
if	una in più, oppure niente	c'è qualcosa da fare solo in un caso
if / else	esattamente due	bisogna fare una cosa o l'altra
if / elif / else	quante ne servono	il caso ha più di due esiti

Non ce ne sono altre. if, elif ed else sono parole chiave riservate, e ognuna apre una clausola: intestazione con i due punti più il blocco rientrato. Un if con due elif e un else è UNA sola istruzione fatta di quattro clausole.

2.4 La condizione è un'espressione booleana di tipo bool

```
SCORTA_MINIMA = 10

prezzo = 49.90

giacenza = 4

# Un confronto PRODUCE un valore, esattamente come 2 + 2 produce 4.
esito = prezzo > 30.00

print(esito)                                # True
print(type(esito))                          # <class 'bool'>

# Operatore '<', operandi `giacenza` e `SCORTA_MINIMA`, risultato bool.
print(giacenza < SCORTA_MINIMA)             # True

# TRABOCCHETTO: un uguale assegna, due uguali confrontano. `giacenza = 4`
#   cambierebbe il valore; `giacenza == 4` si limita a chiedere.
print(giacenza == 4)                        # True
print(giacenza != 4)                        # False
```

3. IF: LA FORMA MINIMA

E l'indentazione che delimita il blocco

3.1 Anatomia di un if: intestazione, due punti, corpo

```
# La soglia della circolare, con un nome. Il giorno in cui passa a 60 si
# cambia questa riga, e la condizione sotto resta quella che è.

SOGLIA_SPEDIZIONE_GRATUITA = 50.00

totale = 62.40

# L'intestazione della clausola: parola chiave, condizione, due punti.
if totale >= SOGLIA_SPEDIZIONE_GRATUITA:
    # Il corpo, rientrato di quattro spazi. Si esegue solo se la condizione
    # è vera. Qui è l'unica riga del blocco.
    print("Spedizione gratuita")

# Colonna 0: fuori dal blocco. Si esegue in ogni caso, anche con totali
# piccoli, e ringraziare chi ha comprato non dipende da quanto ha speso.
print("Grazie per l'acquisto")

# Spedizione gratuita
# Grazie per l'acquisto
```

3.2 L'indentazione delimita il blocco, non lo decora

```
# Una condizione sempre vera, per guardare il blocco e non la condizione.
```

```
if 3 > 2:
```

```
    # Due righe allo stesso rientro: sono lo stesso blocco.
```

```
    print("dentro il blocco")
```

```
    print("anche questa")
```

```
# Il rientro torna a zero: qui il blocco è finito. Nessuna parentesi,
```

```
# nessuna parola chiave lo dichiara: lo dice il margine sinistro.
```

```
print("fuori dal blocco")
```

```
# dentro il blocco
```

```
# anche questa
```

```
# fuori dal blocco
```

Il blocco comincia alla riga dopo i due punti, comprende tutte le righe consecutive rientrate almeno quanto la prima, e finisce alla prima riga che torna al rientro di prima. Guardando il margine sinistro ne capite la struttura senza leggere una parola.

3.3 Quattro spazi per livello: PEP 8 e il TabError

PEP 8: quattro spazi per livello. Non tre, non due, non la tabulazione.

Una tabulazione vale 4 spazi in un editor e 8 in un altro:

voi vedete un programma allineato, il collega ne vede un altro,
Python ne vede un terzo.

Sui file .py PyCharm converte da solo Tab in quattro spazi: è il predefinito.

Da fare adesso, tutti insieme:

View → Active Editor → Show Whitespaces (i menu restano in inglese)
da questo momento vedete i puntini al posto degli spazi

Sembra brutto. Fra due ore vi avrà salvato.

3.4 IndentationError e TabError: i tre rientri sbagliati

IndentationError: expected an indented block after 'if' statement on line 5

-> avete scritto i due punti e non avete rientrato niente

-> il messaggio è generoso: vi dice perfino a quale if si riferisce

IndentationError: unexpected indent

-> uno spazio in più su una riga che non doveva averlo

-> quasi sempre copia-incolla dalla chat, che porta spazi invisibili

TabError: inconsistent use of tabs and spaces in indentation

-> tabulazioni e spazi mescolati nello stesso blocco

-> il peggiore, perché a schermo le due righe sono identiche

Il metodo che funziona sempre: cursore all'inizio del testo, cancellare all'indietro fino al margine, riscrivere gli spazi contandoli. Non provate a sistemare quello che c'è.

3.5 La riga dentro il blocco: la versione corretta

```
SOGLIA_SPEDIZIONE_GRATUITA = 50.00
COSTO_SPEDIZIONE = 6.90

# Diciotto euro: sotto la soglia, quindi la spedizione si paga.
totale = 18.00

if totale >= SOGLIA_SPEDIZIONE_GRATUITA:
    print("Spedizione gratuita")

    # TRABOCCHETTO: questa riga sta DENTRO il blocco, e vale solo per chi
    #   ha diritto alla spedizione gratuita. Spostatela a colonna 0 e lo
    #   sconto viene applicato a tutti. Python non dirà niente: la riga è
    #   sintatticamente perfetta, è solo attaccata al posto sbagliato.
    totale = totale - COSTO_SPEDIZIONE

print(f"Da pagare: {totale:.2f} euro")

# Da pagare: 18.00 euro
```

Diciotto euro di spesa: niente spedizione gratuita, niente sconto. Corretto.

3.5 La riga fuori dal blocco: un altro programma

```
SOGLIA_SPEDIZIONE_GRATUITA = 50.00

COSTO_SPEDIZIONE = 6.90

totale = 18.00

if totale >= SOGLIA_SPEDIZIONE_GRATUITA:
    print("Spedizione gratuita")
# Quattro spazi più a sinistra: la riga è uscita dal blocco e si esegue
# sempre. Nessun errore, nessun avviso, un altro programma.
totale = totale - COSTO_SPEDIZIONE

print(f"Da pagare: {totale:.2f} euro")

# Da pagare: 11.10 euro
```

Un rientro sbagliato di quattro spazi non produce un errore: produce un programma diverso. È l'unico caso in cui Python vi lascia completamente soli. Torna al capitolo 15 e in casa_04.4, dove la stessa riga scala 300 euro da un conto corrente.

3.6 Da provare subito (1): la riga dentro e fuori dal blocco

```
temperatura = 38.5

# Soglia scritta dentro la condizione: qui va bene perché è un frammento da
# tre righe. In un programma vero diventa una costante in cima.
if temperatura > 37.5:
    print("[!] Febbre")

# Colonna 0: la registrazione si stampa per qualunque temperatura.
print("Registrazione completata")
```

Prima scrivete su un foglio cosa vi aspettate, poi eseguite. Poi rientrate di quattro spazi anche l'ultima riga, rilanciate con temperatura = 36.0 e guardate cosa sparisce: quante righe vi escono nei due casi, uno oppure zero?

3.7 if senza else: decisione a una via sola, e pass

```
SOGLIA_RIORDINO = 10
```

```
giacenza = 4
```

```
# Nessun else: su un articolo che sta a posto non c'è niente da dire, e un
```

```
# ramo vuoto sarebbe rumore. Con giacenza = 40 esce solo la riga finale.
```

```
if giacenza < SOGLIA_RIORDINO:
```

```
    print(f"[!] Riordinare: in magazzino {giacenza} pezzi")
```

```
print("Inventario aggiornato")
```

```
# [!] Riordinare: in magazzino 4 pezzi
```

```
# Inventario aggiornato
```

La clausola else è facoltativa. Scrivere else: pass, o un else con dentro un print(""), è rumore: pass significa "qui non si fa niente" e serve solo dove la sintassi pretende un blocco.

4. IF / ELSE

Due rami, uno solo percorso

4.1 if/else: due rami, uno solo eseguito

```
MAGGIORE_ETA = 18

eta = 16

# La condizione è falsa, quindi questo blocco viene saltato per intero.
if eta >= MAGGIORE_ETA:
    print("[OK] Può firmare da solo")
# else non ha condizione: raccoglie tutto quello che l'if ha lasciato fuori.
else:
    print("[!] Serve la firma di un genitore")

# Colonna 0: fuori da tutti e due i rami. Si esegue sempre, ed è il punto
# in cui le due strade si ricongiungono (il disegno del paragrafo 2.2).
print("Pratica registrata")

# [!] Serve la firma di un genitore
# Pratica registrata
```

else significa "in tutti gli altri casi": non ha una condizione, ha solo i due punti. La sua condizione è già scritta, ed è la negazione di quella dell'if.

4.1 Le tre garanzie dell'if/else

Uno dei due rami viene sempre eseguito

Mai tutti e due

Dopo, il programma riprende dalla prima riga non rientrata, in ogni caso

Il giorno in cui la soglia passa da 18 a 16, con if/else cambiate

un numero in un posto solo. Con due if dalle condizioni opposte ne cambiate due, e chi ne cambia uno apre un buco: o non ci finisce nessun caso, o ci finiscono tutti e due.

Analogia - la ricetta con la variante: "se l'impasto è troppo asciutto

aggiungi un cucchiaino d'acqua, altrimenti procedi".

Infornare - il passo dopo - si fa in tutti e due i casi.

4.2 else è una clausola: sta sulla colonna del suo if

```
File "/Users/giulia/corso-python/giorno_04/eta.py", line 8
```

```
else:
```

```
^^^^
```

```
SyntaxError: invalid syntax
```

else non è un'istruzione autonoma: è una clausola, e una clausola deve appartenere a qualcuno. Python, arrivando su quel rientro, non trova nessun if aperto a quella colonna.

if ed else che stanno insieme stanno sulla stessa colonna.

Questo errore è quasi sempre un rientro, non una parola sbagliata. Prima di cercare il refuso, guardate il margine sinistro dell'else e quello dell'if: la verifica si fa con l'occhio sul margine, non leggendo le parole.

4.3 Assegnare nei rami e stampare una volta sola

```
MAGGIORE_ETA = 18

eta = 16

# I rami non stampano: assegnano. Ogni ramo scrive nelle stesse due
# variabili, così dopo la decisione esistono in ogni caso.
if eta >= MAGGIORE_ETA:
    esito = "ammesso"
    nota = "può firmare da solo"
else:
    esito = "non ammesso"
    nota = "serve la firma di un genitore"

# Un solo punto di stampa. Cambiare il formato della ricevuta qui significa
# toccare una riga; con due print dentro i rami sarebbero due, e in 04.8
# sarebbero sette.

print(f"Esito: {esito} - {nota}")

# Esito: non ammesso - serve la firma di un genitore
```

I rami decidono, la stampa stampa. È l'abitudine che oggi vi risparmierà ore.

4.4 La condizione che nega sta nell'if: il rifiuto per primo

```
# Il limite sta in cima, con un nome che dice di che limite si tratta:

# chi legge il programma lo trova senza entrare nella logica.

IMPORTO_MASSIMO_CONTANTI = 999.99

importo = 1200.00

# La condizione che NEGA sta nell'if, e usa > perché il limite stesso è
# ancora ammesso: 999.99 passa, 1000.00 no.

if importo > IMPORTO_MASSIMO_CONTANTI:

    esito = "RIFIUTATO"

    messaggio = "Oltre il limite dei contanti: disporre un bonifico."

else:

    esito = "ACCETTATO"

    messaggio = "Incasso registrato in cassa."

print(f"[{esito}] {messaggio}")

# [RIFIUTATO] Oltre il limite dei contanti: disporre un bonifico.
```

4.5 Oltre i due rami: perché un if in coda sovrascrive

```
VOTO_MINIMO = 18

VOTO_LODE = 30

voto = 30

# Due rami per un problema che di esiti ne ha quattro: promosso, bocciato,
# promosso con riserva, lode. Il programma non sbaglia, dice meno del vero.
if voto >= VOTO_MINIMO:
    esito = "Promosso"
else:
    esito = "Bocciato"

# TRABOCCHETTO: questo secondo if non aggiunge un ramo, SOVRASCRIVE una
# variabile già assegnata. Funziona solo perché sta sotto: spostatelo
# sopra e la lode sparisce, senza che niente lo segnali.
if voto == VOTO_LODE:
    esito = "Promosso con lode"

print(esito)

# Promosso con lode
```

5. IF / ELIF / ELSE

La catena a più rami

5.1 La catena di elif: il primo ramo vero vince

```
punteggio = 95

# 95 soddisfa tutte e tre le condizioni: è maggiore di 90, di 80 e di 70.
# Ne viene eseguita UNA sola, la prima.

if punteggio > 90:
    print("primo ramo")

# TRABOCCHETTO: questi due confronti non sono "valutati e scartati". Non
# vengono proprio eseguiti: la catena è una sola istruzione, e alla prima
# condizione vera salta direttamente alla riga dopo l'else.

elif punteggio > 80:
    print("secondo ramo")

elif punteggio > 70:
    print("terzo ramo")

else:
    print("ramo finale")

# primo ramo
```

elif è la contrazione di else if, "altrimenti, se". Quanti rami vengono eseguiti? Esattamente uno, mai di più. Se nessuna condizione è vera e non c'è else, nessun ramo e il programma prosegue. L'ordine conta, ed è il capitolo 6.

5.2 elif contro if separati: esclusivi o cumulativi

```
# Due caratteristiche che convivono: il collo è di vetro ED è ingombrante.  
  
prodotto_fragile = True  
prodotto_voluminoso = True
```

```
# TRABOCCHETTO: elif dichiara che i casi si escludono. Qui non si escludono  
# affatto, e la seconda etichetta non verrà mai stampata: sul furgone  
# sale un collo di vetro senza avviso di ingombro.  
  
if prodotto_fragile:  
    print("Etichetta FRAGILE")  
  
elif prodotto_voluminoso:  
    print("Etichetta INGOMBRANTE")
```

```
# Etichetta FRAGILE
```

```
# Due if separati: due istruzioni indipendenti, valutate tutte e due. Le  
# etichette si sommano, ed è quello che serve al magazzino.
```

```
if prodotto_fragile:  
    print("Etichetta FRAGILE")  
  
if prodotto_voluminoso:  
    print("Etichetta INGOMBRANTE")
```

```
# Etichetta FRAGILE
```

```
# Etichetta INGOMBRANTE
```

5.2 Lo stesso bivio su un contatore

```
PESO_SUPPLEMENTO = 20.0

peso = 24.5
zona = "ISOLE"

# Un contatore che parte da zero: è lo schema del punteggio del capitolo 14.
supplementi = 0

# Due if separati, perché i due supplementi sono indipendenti: un collo può
# essere pesante, può andare alle isole, e può essere tutte e due le cose.
if peso > PESO_SUPPLEMENTO:
    supplementi = supplementi + 1
if zona == "ISOLE":
    supplementi = supplementi + 1

print(f"Supplementi da applicare: {supplementi}")

# Supplementi da applicare: 2
```

5.3 L'else finale: il ramo che evita il NameError

```
# Una zona che nessuno aveva messo in conto quando la catena è stata scritta.
zona = "ESTERO"

if zona == "NORD":
    tariffa = 6.50
elif zona == "CENTRO":
    tariffa = 7.50
elif zona == "SUD":
    tariffa = 8.50
elif zona == "ISOLE":
    tariffa = 12.00

# TRABOCCHETTO: senza questo else la variabile tariffa non verrebbe creata
# affatto, e la riga di stampa si fermerebbe con NameError: name
# 'tariffa' is not defined. Un messaggio che parla di una variabile a chi
# aveva scritto il nome di una zona.
else:
    tariffa = 0.00
    print(f"[ERRORE] Zona non riconosciuta: {zona}")

print(f"Tariffa: {tariffa:.2f} euro")

# [ERRORE] Zona non riconosciuta: ESTERO
# Tariffa: 0.00 euro
```

5.4 Il triage dell'Aurora: un protocollo tradotto in elif

```
# Le tre soglie del protocollo appeso al muro, con il nome del colore che
# aprono. Sono numeri di reparto, non scelte di chi programma.

DOLORE_ROSSO = 8
DOLORE_GIALLO = 5
DOLORE_VERDE = 2

dolore = 6

# Soglie decrescenti, come in 6.2: si prova prima la condizione più grave.
# Il ramo GIALLO non deve dire "fra 5 e 8", perché se fossimo sopra 8 ci
# saremmo già fermati.

if dolore >= DOLORE_ROSSO:
    colore = "ROSSO"
elif dolore >= DOLORE_GIALLO:
    colore = "GIALLO"
elif dolore >= DOLORE_VERDE:
    colore = "VERDE"
else:
    # Il caso "nessuna delle precedenti", che qui ha un nome suo.
    colore = "BIANCO"

print(f"Codice assegnato: {colore}")
```

```
# Codice assegnato: GIALLO
```

Il protocollo dell'accettazione è già una catena di elif: nessuno l'ha scritto pensando a Python, eppure tradurlo è quasi una trascrizione. Buona parte del lavoro di chi programma è questa.

6. L'ORDINE DELLE SOGLIE

Il difetto che passa il collaudo e lo scopre un cliente

6.1 Soglie crescenti: perché il primo ramo prende tutto

```
spesa = 250.00

# TRABOCCHETTO: le soglie sono in ordine CRESCENTE, cioè dalla condizione
# più larga alla più stretta. 250 supera 50, quindi il primo ramo è vero,
# quindi la catena si ferma qui e i due elif sotto non vengono nemmeno
# valutati. La soglia più bassa si prende tutti i clienti.
if spesa >= 50.00:
    sconto = 5
elif spesa >= 100.00:
    sconto = 10
elif spesa >= 200.00:
    # Riga irraggiungibile: nessuna spesa arriverà mai fin qui.
    sconto = 15
else:
    sconto = 0

print(f"Sconto applicato: {sconto}%")

# Sconto applicato: 5%
```

6.2 Soglie decrescenti con >=: la catena corretta

```
# Le tre soglie in cima: quando il tariffario cambia, si toccano queste.

SOGLIA_BRONZO = 50.00

SOGLIA_ARGENTO = 100.00

SOGLIA_ORO = 200.00


spesa = 250.00


# Soglie in ordine DECRESCENTE: si prova prima la condizione più stretta.
# Ogni ramo ha un confronto solo, perché i rami sopra hanno già escluso
# tutto quello che stava più in alto.

if spesa >= SOGLIA_ORO:

    fascia = "ORO"

    sconto = 15

elif spesa >= SOGLIA_ARGENTO:

    fascia = "ARGENTO"

    sconto = 10

elif spesa >= SOGLIA_BRONZO:

    fascia = "BRONZO"

    sconto = 5

else:

    # TRABOCCHETTO: senza questo ramo una spesa da 20 euro lascerebbe
    # fascia e sconto non assegnate: NameError sulla riga di stampa.

    fascia = "BASE"

    sconto = 0


print(f"Fascia {fascia}, sconto {sconto}%")
```

Stessi dati della catena sbagliata, e stavolta esce Fascia ORO, sconto 15%.

6.2 La regola, e i rami che non si ripetono

Con $\geq$ si parte dalla soglia più alta e si scende.

Con $\leq$ si parte dalla più bassa e si sale.

Quello che non si può fare è mescolare le due direzioni nella stessa catena.

Nella catena scritta bene nessun ramo ripete la condizione dei precedenti

basta `elif spesa >= SOGLIA_ARGENTO;`, perché sopra 200 ci saremmo già fermati
scrivere `and spesa < SOGLIA_ORO` funziona, ma è aria fritta
e una condizione in più è una condizione che si può sbagliare

Analogia - la dogana: i controlli si fanno dal più stringente al meno

stringente. Se metti per primo il controllo largo, quello stretto non
scatta mai: passa tutto dal varco grande.

6.3 Il bordo della soglia: > contro >=

```
SOGLIA_BRONZO = 50.00

# Il valore esatto della soglia: l'unico caso in cui i due operatori danno
# risposte diverse. Sotto e sopra si comportano identici.
spesa_al_bordo = 50.00

# "da 50 euro": il 50 è dentro, serve >=.    -> True
# "oltre 50 euro": il 50 è fuori, serve >.    -> False
print(spesa_al_bordo >= SOGLIA_BRONZO)
print(spesa_al_bordo > SOGLIA_BRONZO)
print(spesa_al_bordo == SOGLIA_BRONZO)

# True
# False
# True

# 49.99 -> BASE con tutti e due.  50.00 -> BRONZO con >=, BASE con >.
# 50.01 -> BRONZO con tutti e due.
```

6.4 Da provare subito (2): la catena sbagliata sui bordi

Spesa	Sconto atteso	Sconto ottenuto
30.00	0%	?
50.00	5%	?
99.99	5%	?
100.00	10%	?
250.00	15%	?

Copiate la catena sbagliata del paragrafo 6.1 e provatela con questi cinque valori, uno alla volta. La terza colonna si compila a mano PRIMA di eseguire. Poi scrivete in chat quanti di questi cinque danno comunque la risposta giusta anche con la catena sbagliata.

6.5 Fasce ISEE con $\leq$ crescente: le soglie come costanti

```
FASCIA_ISEE_1 = 8000      # fino a 8.000      -> quota 15.00
FASCIA_ISEE_2 = 16000     # da 8.001 a 16.000 -> quota 30.00
FASCIA_ISEE_3 = 28000     # da 16.001 a 28.000 -> quota 45.00

QUOTA_FASCIA_1 = 15.00
QUOTA_FASCIA_2 = 30.00
QUOTA_FASCIA_3 = 45.00
QUOTA_FASCIA_4 = 60.00   # oltre 28.000

isee = 16000              # valore di collaudo sul bordo della fascia 2

# Con  $\leq$  si parte dalla soglia più BASSA e si sale: è la direzione opposta
# a quella del paragrafo 6.2, ed è corretta per lo stesso motivo.

if isee <= FASCIA_ISEE_1:
    quota = QUOTA_FASCIA_1
elif isee <= FASCIA_ISEE_2:
    quota = QUOTA_FASCIA_2
elif isee <= FASCIA_ISEE_3:
    quota = QUOTA_FASCIA_3
else:
    quota = QUOTA_FASCIA_4

# TRABOCCHETTO: 16.000 esatti finiscono in fascia 2, non in fascia 3. La
# tabella dice "da 8.001 a 16.000": il 16.000 è incluso, quindi  $\leq$  e non
# <. È il dettaglio che si chiede all'ufficio PRIMA di scrivere il codice.

print(f"Quota mensile: {quota:.2f} euro")
```

7. GLI OPERATORI DI CONFRONTO IN AZIONE

Tre sorprese sulle stringhe, due sui numeri

7.1 I sei operatori di confronto: == confronta, = assegna

Operatore	Si legge	Esempio vero
==	è uguale a	"SUD" == "SUD"
!=	è diverso da	18 != 21
<	è minore di	2.5 < 10
>	è maggiore di	100 > 99.99
<=	è minore o uguale a	5 <= 5
>=	è maggiore o uguale a	5 >= 5

Sei, e non ce n'è un settimo: il "fra due valori" si costruisce con questi al capitolo 10. Un uguale assegna, due uguali confrontano, e Python vi aiuta: if voto = 30: dà `SyntaxError: invalid syntax. Maybe you meant '==' or ':=' instead of '='`?

7.2 Confronto fra stringhe: ordine lessicografico

```
# 1. e 2.: == fra stringhe confronta i caratteri uno per uno. Una maiuscola
```

```
# diversa o uno spazio invisibile bastano a far uscire False.
```

```
print("sud" == "SUD")           # False
```

```
print("si " == "si")           # False
```

```
# 3. Lo stesso dato, normalizzato prima del confronto: adesso True.
```

```
print("si ".strip().lower() == "si")  # True
```

```
# TRABOCCHETTO: con < e > su stringhe si confronta l'ordine dei caratteri,
```

```
# non il significato: tutte le maiuscole vengono prima delle minuscole,
```

```
# e "10" precede "9" perché il carattere 1 viene prima del carattere 9.
```

```
print("Zurigo" < "ancona")        # True
```

```
print("10" < "9")                 # True
```

```
# Ordine lessicografico: si confronta da sinistra e ci si ferma al primo
```

```
# carattere diverso; se uno dei due finisce prima, il più corto è il minore.
```

```
print("ancona" < "ancora")        # True   n viene prima di r
```

```
print("via" < "viale")            # True   "via" finisce
```

```
print("" < "a")                   # True   la vuota precede tutto
```

7.3 Confronto fra tipi diversi: TypeError con >=

```
Traceback (most recent call last):
```

```
File "/Users/giulia/corso-python/giorno_04/eta.py", line 3, in <module>
```

```
    if eta >= 18:
```

```
        ^^^^^^^
```

```
TypeError: '>=' not supported between instances of 'str' and 'int'
```

Manca la conversione: `eta` è arrivato da `input()` ed è ancora una stringa. Con `>=`, `>`, `<` e `<=` Python solleva l'eccezione, perché non esiste nessun criterio per dire se "30" sia maggiore o minore di 30.

Almeno qui ve ne accorgete: il programma si ferma e vi dice esattamente che cosa non torna. Il caso gemello della prossima slide è molto peggiore.

7.3 Con ==: nessun errore, e una condizione sempre falsa

```
# A schermo sono identici. In memoria sono due oggetti di tipo diverso:
# "30" è una stringa di due caratteri, 30 è un numero intero.

testo_eta = "30"

# TRABOCCHETTO: questa riga non solleva niente e stampa False. La domanda
# "sono lo stesso valore?" una risposta ce l'ha sempre, ed è no. Una
# condizione così resta falsa per sempre, in silenzio.

print(testo_eta == 30)          # False

# Con la conversione esplicita i due operandi sono dello stesso tipo.

print(int(testo_eta) == 30)     # True
```

Con == il criterio c'è, e il verdetto è False: vero e inutile. È il blocco 5 di casa_04.4, e la differenza fra i due comportamenti è voluta, non un'incoerenza di Python.

7.4 Numeri con la virgola: mai ==, si usa una tolleranza

```
# Quanto due decimali possono distare per essere considerati uguali. Il
# valore si sceglie in base al dato: qui un millesimo.
TOLLERANZA = 0.001

totale = 0.1 + 0.2

# TRABOCCHETTO: la prima riga mostra il numero vero, che non è 0.3. La
# seconda è il confronto che in aula tutti danno per vero e che stampa
# False. Non è un difetto di Python: è come sono fatti i float dentro
# qualunque computer.
print(totale)                # 0.30000000000000004
print(totale == 0.3)         # False

# La forma da usare: abs() toglie il segno alla differenza, così la stessa
# riga funziona sia che il totale ecceda, sia che manchi.
print(abs(totale - 0.3) < TOLLERANZA)    # True
```

7.5 Pronto soccorso: quale confronto per quale domanda

Volete sapere se...	Si scrive	Attenzione a
due testi sono lo stesso testo	<code>a.strip().lower() == b</code>	spazi e maiuscole
un numero supera una soglia	<code>valore >= SOGLIA</code>	<code>></code> contro <code>>=</code> sul bordo
un valore sta in un intervallo	<code>MIN <= valore <= MAX</code>	capitolo 10
due decimali sono quasi uguali	<code>abs(a - b) < TOLLERANZA</code>	mai con <code>==</code>
un testo contiene una parola	<code>"gratis" in messaggio</code>	maiuscole, capitolo 14
un campo è stato compilato	<code>if campo:</code>	capitolo 11

Sei domande che vi farete oggi pomeriggio, con la forma da scrivere e la trappola che nasconde. Tenetela aperta durante il laboratorio.

8. AND, OR, NOT

Gli operatori logici

8.1 and: due condizioni vere insieme, il caso antifrode

```
IMPORTO_ALTO = 3000.00

importo = 4500.00

# Un dato già booleano: arriva dall'archivio dei dispositivi del cliente e
# vale True o False. Non serve confrontarlo con niente.
dispositivo_noto = False

# Due indizi legati da and: il blocco scatta solo se sono veri tutti e due.
#   importo >= IMPORTO_ALTO  ->  True   (4500 supera la soglia)
#   not dispositivo_noto     ->  True   (not ribalta il False di partenza)
# TRABOCCHETTO: `not dispositivo_noto` non è un `== False` scritto male. È
#   la stessa cosa, e si legge come si direbbe a voce.
if importo >= IMPORTO_ALTO and not dispositivo_noto:
    print("[!] Pagamento bloccato: verifica necessaria")
else:
    print("[OK] Pagamento autorizzato")

# [!] Pagamento bloccato: verifica necessaria
```

8.2 Le tabelle di verità di and, or, not

A	B	A and B	A or B	not A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

and è vero solo se lo sono tutte e due; or è vero se lo è almeno una; not ribalta e prende un operando solo. L'or dell'informatica è INCLUSIVO: è vero anche quando sono veri tutti e due. È una differenza con l'italiano parlato, ed è la prima causa di condizioni troppo larghe.

8.3 Precedenza: not, poi and, poi or

```
a = True
```

```
b = True
```

```
c = False
```

```
# Senza parentesi vince and, che lega più stretto: a or (b and c) = True.
```

```
print(a or b and c)      # True
```

```
# Con le parentesi l'or si risolve per primo: (a or b) and c = False.
```

```
# Stesse tre variabili, due risultati opposti.
```

```
print((a or b) and c)    # False
```

*Si chiama precedenza degli operatori, ed è la stessa cosa che in aritmetica fa valere $2 + 3 * 4$ come 14 e non come 20. and è la moltiplicazione, or è la somma.*

8.3 La precedenza su un caso vero: due politiche

```
IMPORTO_ALTO = 3000.00

importo = 500.00

dispositivo_noto = True

paese_abituale = False

# TRABOCCHETTO: senza parentesi Python raggruppa (importo alto AND
#   dispositivo ignoto) OR paese insolito. Basta il paese insolito e il
#   blocco scatta, su qualunque importo e da qualunque telefono.
esito_senza = importo >= IMPORTO_ALTO and not dispositivo_noto or not paese_abituale

# Con le parentesi l'importo alto diventa obbligatorio, e i due indizi
# sospetti valgono come alternative fra loro. È la politica voluta.
esito_con = importo >= IMPORTO_ALTO and (not dispositivo_noto or not paese_abituale)

print(esito_senza)    # True
print(esito_con)      # False
```

8.4 reparto == "a" or "b": la condizione sempre vera

```
reparto = "contabilità"

# TRABOCCHETTO: Python legge due operandi separati, non "uguale a uno dei
# due". Il primo è il confronto reparto == "magazzino", falso. Il secondo
# è la stringa "vendite" da sola: non un confronto, un valore, e un testo
# non vuoto vale True. False or True fa True, per qualunque reparto.
if reparto == "magazzino" or "vendite":
    print("Accesso consentito")
else:
    # Riga irraggiungibile. Non ci arriverà mai nessuno.
    print("Accesso negato")

# Accesso consentito

# Con reparto = "portineria" viene consentito anche a quella.
# Viene consentito sempre.
```

8.4 Smontarla: or non restituisce un booleano

```
reparto = "contabilità"

# 1. e 2.: i due operandi guardati da soli. Falso il primo, vero il secondo.
print(reparto == "magazzino")          # False
print(bool("vendite"))                  # True

# 3. L'espressione intera. Guardate bene: non stampa True, stampa vendite.
print(reparto == "magazzino" or "vendite") # vendite

# La forma corretta ripete il confronto per esteso, e con "contabilità"
# stampa Accesso negato.
if reparto == "magazzino" or reparto == "vendite":
    print("Accesso consentito")
else:
    print("Accesso negato")
```

8.5 Negare una condizione composta: not e le parentesi

```
DOLORE_ROSSO = 8
```

```
cosciente = True
```

```
dolore = 9
```

```
# Si legge: "se non è cosciente OPPURE il dolore è almeno otto". Con or  
# basta un indizio, e il paziente incosciente senza dolore resta dentro.  
if not cosciente or dolore >= DOLORE_ROSSO:  
    print("Codice ROSSO")
```

```
# Codice ROSSO
```

```
# TRABOCCHETTO: quando si negano due condizioni insieme le parentesi sono  
# obbligatorie. not (documento_valido and foto_conforme) si legge "non è  
# vero che ci sono tutte e due le cose"; senza parentesi,  
# not documento_valido and foto_conforme dice tutt'altro.
```

8.5 Le cinque forme da avere in tasca

Volete dire	Si scrive
tutte e due le cose	a and b
almeno una delle due	a or b
nessuna delle due	not a and not b
non tutte e due	not (a and b)
una sì e l'altra no	(a and not b) or (b and not a)

Le due righe di mezzo sono quelle che si scambiano: "nessuna delle due" e "non tutte e due" sono cose diverse. Per chi vuole andare oltre: not (A and B) equivale a (not A) or (not B), e not (A or B) a (not A) and (not B). Si chiamano leggi di De Morgan.

9. LA VALUTAZIONE PIGRA

Il cortocircuito di and e or

9.1 Cortocircuito: and al primo falso, or al primo vero

```
# A and B   prima parte falsa -> la seconda NON viene valutata
# A and B   prima parte vera  -> la seconda viene valutata
# A or  B   prima parte vera  -> la seconda NON viene valutata
# A or  B   prima parte falsa -> la seconda viene valutata

ETA_MINIMA = 18

# Una stringa che int() non saprebbe convertire: da sola, quella chiamata
# farebbe terminare il programma con l'eccezione ValueError.
testo_eta = "ventotto"

# 1. and: il primo operando è False, il risultato è già deciso, il secondo
# operando NON viene eseguito. La conversione non avviene mai.
print(testo_eta.isdecimal() and int(testo_eta) >= ETA_MINIMA)    # False

# 2. or: il primo operando è True (not su False), il risultato è già
# deciso, e di nuovo il secondo non viene eseguito.
print(not testo_eta.isdecimal() or int(testo_eta) >= ETA_MINIMA) # True
```

9.2 La guardia che protegge la conversione

```
ETA_MINIMA = 18

testo_eta = "diciotto"

# L'ordine degli operandi è la cosa importante di tutta la riga. Prima la
# guardia "è fatto di sole cifre?", poi la domanda "quanto vale?". La
# seconda viene eseguita solo se la prima ha risposto sì.
if testo_eta.isdecimal() and int(testo_eta) >= ETA_MINIMA:
    print("[OK] Maggiorennne")
else:
    # Qui finiscono due casi diversi: il dato non numerico e il minorennne.
    # Per questo il messaggio dice tutte e due le cose.
    print("[!] Età non valida o non sufficiente")

# [!] Età non valida o non sufficiente

# Invertendo i due operandi:
# ValueError: invalid literal for int() with base 10: 'diciotto'
```

9.2 Due guardie incatenate: presenza, poi forma

```
# L'utente ha battuto tre spazi e via. Dopo .strip() resta la stringa
# vuota: tipo str, lunghezza zero.
testo_protocollo = "   "
protocollo = testo_protocollo.strip()

# Due guardie incatenate. La prima è la forma breve del capitolo 11 e vale
# False sulla stringa vuota: il cortocircuito scatta lì, e .isdecimal() non
# viene mai chiamato.
if protocollo and protocollo.isdecimal():
    print(f"Protocollo {protocollo} preso in carico")
else:
    print("[ERRORE] Indicare il protocollo, per esempio 00041287.")

# [ERRORE] Indicare il protocollo, per esempio 00041287.
```

9.2 Con or la guardia va negata

```
SCONTO_MASSIMO = 30

testo_sconto = "trenta"

# TRABOCCHETTO: qui la guardia è NEGATA, e non è un capriccio. L'or si
#   ferma sul primo vero, quindi per proteggere int() il primo operando
#   deve diventare vero proprio quando il dato è inutilizzabile.
if not testo_sconto.isdecimal() or int(testo_sconto) > SCONTO_MASSIMO:
    print(f"[ERRORE] Lo sconto va indicato in cifre, da 0 a {SCONTO_MASSIMO}.")
else:
    print(f"[OK] Sconto del {testo_sconto} per cento applicato")

# [ERRORE] Lo sconto va indicato in cifre, da 0 a 30.
```

Con and va davanti la guardia in positivo ("è un numero"), con or la guardia negata ("non è un numero"). Togliete quel not e invertite i due operandi: stessa logica per un essere umano, ValueError per la macchina.

9.3 Ordine degli operandi: prima quello che scarta di più

C'è anche un motivo di efficienza, che oggi non si sente e più avanti sì.

if importo >= IMPORTO_RILEVANTE and paese != "IT":

il primo pezzo scarta da solo la gran parte dei bonifici

per quelli, il confronto sul paese non viene mai eseguito

Su un bonifico non cambia niente. Quando lo stesso controllo girerà su cinquemila bonifici uno dietro l'altro, la differenza si misura.

Per chi vuole andare oltre

and e or restituiscono uno dei due operandi, non un booleano (8.4)

fuori da un if diventa una scorciatoia: "" or "predefinito" vale

"predefinito". Sappiatela riconoscere quando la leggete,

e non usatela quando scrivete.

10. CONFRONTI CONCATENATI

La forma che Python permette e quasi nessun altro

10.1 18 <= eta < 65: il confronto concatenato

```
eta = 34
```

```
# Una riga, due confronti, una sola variabile in mezzo. Equivale a  
# `eta >= 18 and eta < 65`, e si legge molto meglio.
```

```
if 18 <= eta < 65:
```

```
    print("Fascia in età da lavoro")    # Fascia in età da lavoro
```

```
ETA_MINIMA_LAVORO = 18
```

```
ETA_PENSIONE = 65
```

```
eta = 70
```

```
# 1. e 2.: le due scritture danno lo stesso valore di verità. Sono la  
# stessa espressione, scritta in due modi.
```

```
print(ETA_MINIMA_LAVORO <= eta < ETA_PENSIONE)    # False
```

```
print(eta >= ETA_MINIMA_LAVORO and eta < ETA_PENSIONE)    # False
```

```
# 3. e 4.: la stessa condizione smontata nei suoi due pezzi. È il metodo di  
# diagnosi: il primo è vero, il secondo no, ed è lì che sta la risposta.
```

```
print(ETA_MINIMA_LAVORO <= eta)    # True
```

```
print(eta < ETA_PENSIONE)    # False
```

10.2 voto > 18 and < 30 : come si dice, come si scrive

Come si dice	Come si scrive
"fra 18 e 65, estremi inclusi"	$18 \leq \text{eta} \leq 65$
"da 18 a 64"	$18 \leq \text{eta} < 65$
"sopra 18 e sotto 65"	$18 < \text{eta} < 65$
"fuori dall'intervallo"	$\text{not } (18 \leq \text{eta} \leq 65)$

In italiano si dice "se il voto è fra 18 e 30". Tradotto alla lettera, if voto > 18 and < 30 : dà `SyntaxError: invalid syntax`, con il segnalino sotto il $<$ orfano: Python non ha modo di indovinare che quel < 30 si riferisce ancora a voto.

10.2 Operatori in direzioni opposte: nessun errore

```
eta = 34
```

```
# 1. e 2.: operatori nella stessa direzione. Dicono la stessa cosa, letta
```

```
# da sinistra e da destra. Tutte e due descrivono un intervallo.
```

```
print(18 < eta < 65)      # True
```

```
print(65 > eta > 18)      # True
```

```
# TRABOCCHETTO: qui le punte guardano da parti opposte. Si legge "18 è
```

```
# minore dell'età E l'età è maggiore di 65": non è un intervallo, è una
```

```
# condizione vera solo sopra i 65 anni. Python non ha niente da ridire,
```

```
# perché sintatticamente la riga è perfetta.
```

```
print(18 < eta > 65)      # False
```

In un confronto concatenato i due operatori devono puntare nella stessa direzione: o tutti e due verso destra (<, <=) o tutti e due verso sinistra (>, >=). Se in una riga vedete un < e un >, quella riga non descrive un intervallo, qualunque cosa avesse in mente chi l'ha scritta.

10.3 Validare un intervallo: $\text{MIN} \leq \text{valore} \leq \text{MAX}$

```
# I due estremi come costanti: la scala del dolore è una convenzione del
# reparto, non un numero da scrivere dentro una condizione.

DOLORE_MINIMO = 0
DOLORE_MASSIMO = 10

dolore = 12

# Estremi INCLUSI: due `<=`, perché 0 e 10 sono valori legittimi. Con `<`
# il paziente che dichiara dieci si vedrebbe rifiutare il modulo.
if DOLORE_MINIMO <= dolore <= DOLORE_MASSIMO:
    print(f"Dolore registrato: {dolore}")
else:
    # Il messaggio ripesci le costanti: se la scala cambia, cambia da sola.
    print(f"[ERRORE] Indicare un valore da {DOLORE_MINIMO} a {DOLORE_MASSIMO}")

# [ERRORE] Indicare un valore da 0 a 10
```

10.4 Tre o più termini: dove smette di leggersi

Si possono concatenare più di due confronti:

apertura $\leq$ ora $\leq$ chiusura funziona benissimo

Ma oltre tre termini la riga smette di essere leggibile.

Vale anche $5 == 5 == 5$, che significa "il primo è uguale al secondo e il secondo al terzo": è corretto, ma in aula genera più domande di quante ne risolva.

Se avete tre valori da confrontare, scriveteli con un `and` esplicito e dormite tranquilli.

10.5 Fasce con intervalli espliciti: due forme legittime

La catena del paragrafo 6.2 si può riscrivere con gli intervalli dichiarati:

```
elif SOGLIA_ARGENTO <= spesa < SOGLIA_ORO:
```

```
al posto del più corto elif spesa >= SOGLIA_ARGENTO:
```

A favore

richiede meno di aver capito che i rami precedenti hanno già escluso il resto
regge meglio la rilettura di un collega che non ha seguito il corso

Contro

ogni soglia compare due volte

senza le costanti in cima sarebbe un difetto in agguato al primo ritocco

Nessuna delle due è "quella giusta": si sceglie in base a chi manterrà il programma.

11. VALORE DI VERITÀ

Truthy e falsy

11.1 Il valore di verità: la conversione implicita a bool

```
nome = "Giulia"
```

```
# Nessun operatore di confronto qui dentro: la condizione è una variabile
```

```
# nuda. Python la converte a bool da solo, e un testo non vuoto vale True.
```

```
if nome:
```

```
    print(f"Buongiorno, {nome}")
```

```
else:
```

```
    # TRABOCCHETTO: qui si finisce con nome = "", non con nome mancante.
```

```
    # Una variabile che non esiste non è falsa: solleva NameError.
```

```
    print("[!] Il campo nome è obbligatorio")
```

```
# Buongiorno, Giulia
```

```
# Con nome = "" esce il secondo messaggio.
```

Ogni valore ha un valore di verità: messo dentro un if viene convertito in True o in False. È una conversione IMPLICITA, l'opposto di quella esplicita di int(), float() o bool(). I valori che si convertono in True si dicono truthy, gli altri falsy: if nome: e if bool(nome): sono la stessa cosa.

11.2 I valori falsy: False, 0, 0.0, stringa vuota

Valore	In un if vale	Nota
False	falso	ovvio
0	falso	lo zero intero
0.0	falso	lo zero con la virgola
""	falso	la stringa vuota
" "	VERO	uno spazio è un carattere
"0"	VERO	è un testo, e non è vuoto
ogni altro numero	vero	anche i negativi
ogni altro testo	vero	anche "False"

L'elenco dei falsy, con quello che sapete oggi, è cortissimo. Le due righe in maiuscolo sono quelle che sorprendono, e sono il motivo per cui `.strip()` va messo prima del controllo di presenza.

11.2 bool(): la conversione esplicita, a voce alta

```
campo = "   "

# TRABOCCHETTO: tre spazi sono tre caratteri. La stringa non è vuota,
# quindi è truthy, quindi un campo lasciato in bianco passerebbe il
# controllo di presenza. È per questo che .strip() viene prima.
print(bool(campo))          # True
print(bool(campo.strip()))  # False

# "0" è un testo di un carattere: truthy. Lo 0 intero è falsy. Stesso
# simbolo a schermo, due tipi diversi, due valori di verità opposti.
print(bool("0"))            # True
print(bool(0))              # False

# -1 è truthy: la regola non è "positivo uguale vero", è "diverso da zero".
print(bool(-1))             # True

# "False" fra virgolette è un testo non vuoto: Python guarda se la stringa
# ha dei caratteri, non che cosa significano per un lettore umano.
print(bool("False"))        # True
```

bool() fa a voce alta quello che l'if fa in silenzio: usatela ogni volta che avete un dubbio su un valore. Con sconto = 0.0 un if sconto: stampa "Nessuno sconto", e qui è quello che vogliamo; al paragrafo 11.4 la stessa scorciatoia diventa un difetto.

11.3 if not campo: il controllo di presenza su un testo

```
# .strip() prima del controllo: senza, tre spazi digitati per sbaglio
# passerebbero per un protocollo valido (paragrafo 11.2).
protocollo = input("Numero di protocollo: ").strip()

# not su un valore truthy dà False, su uno falsy dà True. Qui si legge
# "se il protocollo non c'è", ed è la frase che c'era da scrivere.
if not protocollo:
    print("[ERRORE] Il numero di protocollo è obbligatorio")
else:
    print(f"Pratica {protocollo} presa in carico")
```

if not protocollo: si legge "se il protocollo non c'è". La forma lunga if protocollo == "": dice la stessa cosa in modo più macchinoso. È il caso del modulo online in cui qualcuno preme Invio senza scrivere niente, e succede più spesso di quanto immaginate.

11.3 La variabile errore che parte vuota

```
# La stringa vuota è il valore "non è successo niente di storto": è falsy,
# e questo permette di interrogarla alla fine senza nessun confronto.
errore = ""
quantita = 0

# Ogni controllo scrive dentro errore. Qui ce n'è uno solo, in 04.4 sono
# quattro in catena, ma lo schema non cambia.
if quantita == 0:
    errore = "La quantità deve essere almeno 1 pezzo."

# TRABOCCHETTO: questo è il testo dell'errore, non un flag vero/falso.
# Funziona perché una stringa piena è truthy: non serve una seconda
# variabile che dica "c'è stato un problema".
if errore:
    print(f"[ERRORE] {errore}")
else:
    print("[OK] Ordine registrato")

# [ERRORE] La quantità deve essere almeno 1 pezzo.
```

È l'altro uso che vi serve oggi stesso, nell'esercizio 04.4: una variabile che parte vuota e viene riempita solo se qualcosa non va.

11.4 Lo zero è un dato: sui numeri il confronto esplicito

```
# Zero pezzi in magazzino: un dato che c'è, è aggiornato, e vale zero.
giacenza = 0

# TRABOCCHETTO: la condizione qui sta rispondendo alla domanda sbagliata.
# Chi l'ha scritta voleva chiedere "il dato è arrivato?", Python risponde
# a "il numero è diverso da zero?". Sui numeri le due domande non
# coincidono, e l'articolo esaurito viene scambiato per un dato mancante.
if giacenza:
    print(f"In magazzino: {giacenza} pezzi")
else:
    print("Dato di giacenza non disponibile")

# Dato di giacenza non disponibile
# Il messaggio è falso: l'articolo è esaurito, e quella è un'informazione
# preziosa. Qui si scrive if giacenza > 0: con un else che dice
# [!] Articolo esaurito.
```

La forma breve if valore: si usa sui TESTI, per dire "il campo è stato compilato". Sui numeri si scrive sempre il confronto esplicito, perché lo zero è un numero come gli altri.

12. ANNIDAMENTO DELLE CONDIZIONI

Massimo due livelli, e non è una regola del linguaggio

12.1 if esterno e if interno: un rientro in più

```
TARIFFA_ISOLE = 12.00

PESO_ISOLE_PESANTE = 20.0

SUPPLEMENTO_ISOLE_PESANTE = 5.00


zona = "ISOLE"

peso = 24.5

totale = TARIFFA_ISOLE      # i supplementi si sommano sopra la base


# 1. if esterno, colonna 0. Il suo blocco sta a 4 spazi.

if zona == "ISOLE":

    print("Spedizione verso le isole")

    # 2. if interno, colonna 4: la domanda sul peso viene posta SOLO sulle

    #     spedizioni alle isole. Il suo blocco sta a 8 spazi.

    if peso > PESO_ISOLE_PESANTE:

        totale = totale + SUPPLEMENTO_ISOLE_PESANTE

        print("[!] Supplemento collo pesante applicato")


# TRABOCCHETTO: questa riga è a colonna 0, fuori da tutti e due i blocchi.

#     Si esegue sempre, ed è quello che vogliamo: il totale va stampato anche

#     per il NORD, dove nessuno dei due if è scattato.

print(f"Totale: {totale:.2f} euro")
```

12.2 Il limite dei due livelli: una convenzione

In questo corso non si va oltre due livelli di annidamento.

Non è una regola del linguaggio: è una regola di leggibilità,
e in azienda la trovate scritta in quasi tutte le convenzioni interne.

Con tre livelli il blocco più interno sta a dodici spazi dal margine,
e per sapere in quali condizioni viene eseguito bisogna risalire
tre righe diverse: nessuno lo fa.

Il conto dei rientri, sempre lo stesso: 0, 4, 8.

12.3 Appiattare l'annidamento con and: quando è equivalente

```
PESO_ISOLE_PESANTE = 20.0
SUPPLEMENTO_ISOLE_PESANTE = 5.00

zona = "ISOLE"
peso = 24.5

# Il valore di ripiego: se nessuna condizione scatta, il supplemento resta
# zero e la riga finale ha comunque qualcosa da stampare.
supplemento = 0.00

# Un solo livello, una sola condizione composta. L'and porta con sé il
# cortocircuito del capitolo 9: sul NORD il peso non viene nemmeno
# guardato, esattamente come nella versione annidata.
if zona == "ISOLE" and peso > PESO_ISOLE_PESANTE:
    supplemento = SUPPLEMENTO_ISOLE_PESANTE

print(f"Supplemento: {supplemento:.2f} euro")

# Supplemento: 5.00 euro

# Appiattare con and      -> dentro l'if interno c'è una riga sola
# Tenere l'annidamento   -> l'if esterno fa anche altro
#                          -> i due if hanno rami else diversi
#                          -> la condizione unita supera gli 88 caratteri
```

Stesso risultato: una riga invece di due, e la condizione si legge tutta insieme. Qui l'appiattita è preferibile; le quattro righe in fondo dicono quando non lo è.

12.4 Quando l'annidamento resta: i due livelli

```
COSTO_CERTIFICATO = 5.50
MARCA_DA_BOLLO = 16.00

scelta = "1"
bollo = "si"

# Le due variabili di uscita, entrambe inizializzate prima della decisione:
# qualunque ramo venga percorso, alla fine esistono tutte e due.

importo = 0.00
errore = ""

# Livello 1: quale pratica. Livello 2: come si paga quella pratica.
if scelta == "1":
    if bollo == "si":
        importo = COSTO_CERTIFICATO + MARCA_DA_BOLLO
    elif bollo == "no":
        importo = COSTO_CERTIFICATO
    else:
        # Terzo esito del ramo interno: la risposta non è né si né no.
        errore = "Rispondere si oppure no."
else:
    errore = "Voce di menu non valida."
```

Serve quando il blocco esterno ha una vita sua. In 04.8 succede esattamente questo: la voce di menu decide QUALE domanda fare, e solo dopo aver ricevuto la risposta la si può validare.

12.4 Quando l'annidamento resta: perché l'and non basta

```
# TRABOCCHETTO: qui l'and non scioglierebbe l'annidamento. Una condizione
#   composta ha due esiti, e il ramo interno ne ha tre: fondendoli si perde
#   la distinzione fra "voce sbagliata" e "risposta sul bollo sbagliata".
if errore:
    print(f"[ERRORE] {errore}")
else:
    print(f"Importo dovuto: {importo:.2f} euro")

# Importo dovuto: 21.50 euro
```

Due livelli, e bastano. Il criterio non è "quante condizioni ci sono": è quanti esiti distinti deve produrre il ramo interno.

12.5 Il terzo livello come sintomo: serve un nome

Se vi accorgete di aver scritto tre if uno dentro l'altro,
quasi sempre il problema non è l'indentazione:
è che state facendo fare a un pezzo di programma troppe cose insieme.

La soluzione professionale è dare un nome a quel pezzo e metterlo da parte.
Si chiama funzione, e arriva più avanti in queste giornate.

Da lì in poi

il triage del 04.3 diventerà `assegna_colore(dolore, temperatura)`
il classificatore del 04.7 diventerà `classifica_candidato(...)`
con una riga sola nel programma principale

Oggi la logica di decisione resta tutta dentro `main()`: è voluto,
ed è l'ultima giornata in cui lo sarà.

13. VALIDARE L'INPUT PRIMA DI USARLO

Prima si guarda, poi si usa

13.1 Validazione preventiva: prima si controlla

```
# 1. Lettura e normalizzazione. Il dato è ancora una stringa, e nessuno
#    l'ha ancora convertito: è questa la differenza con il Giorno 03.
testo_quantita = input("Quantità: ").strip()

# 2. La guardia. .isdecimal() risponde True solo su sequenze di sole cifre.
if testo_quantita.isdecimal():
    # 3. La conversione sta DENTRO il ramo buono: qui non può fallire,
    #    perché la guardia ha già scartato tutto il resto.
    quantita = int(testo_quantita)
    print(f"Registrati {quantita} pezzi")
else:
    # TRABOCCHETTO: qui non si nomina l'eccezione. Chi sta alla cassa non
    #    deve sapere cosa sia un ValueError: gli serve sapere cosa fare.
    print("[ERRORE] La quantità deve essere un numero intero.")
    print("Esempio: 3. Ripetere l'operazione.")
```

Con "tre" in ingresso escono le due righe di errore: nessun traceback, nessuna riga rossa, nessun cliente in fila davanti a uno schermo bloccato. È il post-it di stamattina, chiuso.

13.2 I quattro controlli, in quest'ordine

#	Controllo	Domanda	Strumento
1	presenza	il campo è stato compilato?	if not testo:
2	forma	è del tipo giusto?	.isdecimal(), .replace()
3	intervallo	il valore è sensato?	MIN <= v <= MAX
4	valori ammessi	è uno di quelli previsti?	== unito da or

Sono quattro, in sequenza, perché ciascuno presuppone il precedente. L'ordine non è negoziabile: non si può chiedere "quanto vale" a un campo che non è ancora stato riconosciuto come numero. È la logica della valutazione pigra del capitolo 9, applicata a una catena di elif.

13.3 Controllo 1 — presenza: `if not campo.strip()`:

Si scrive così, e dentro il ramo va il messaggio d'errore:

```
if not cognome.strip():  
    print("[ERRORE] Il cognome è obbligatorio.")
```

`.strip()` prima del controllo, sempre.

senza, tre spazi digitati per sbaglio passerebbero per un cognome valido
perché " " è un testo non vuoto, e quindi vero (paragrafo 11.2)

È il controllo che viene per primo perché tutti gli altri
presuppongono che nel campo ci sia qualcosa.

13.4 Controllo 2 — forma: `.isdecimal()` contro `.isdigit()`

Il metodo `.isdecimal()` si chiama su una stringa e restituisce un bool.

Accetta una cosa sola: una sequenza di almeno una cifra, e nient'altro.

```
print("42".isdecimal())      # True
```

TRABOCCHETTO: cinque casi che tutti danno per buoni e che tornano False.

Stringa vuota, punto decimale, segno meno, spazio in coda, lettere.

È da qui che nasce la regola di mettere `.strip()` prima.

```
print("").isdecimal()       # False
```

```
print("3.5".isdecimal())    # False
```

```
print("-5".isdecimal())     # False
```

```
print("42 ".isdecimal())    # False
```

```
print("quaranta".isdecimal()) # False
```

Cinque False su sei: `.isdecimal()` è severissimo, ed è la qualità che ci serve. Perché non `.isdigit()`? Sembrano identici e quasi sempre lo sono, ma `"2".isdigit()` è True mentre `"2".isdecimal()` è False, e `int("2")` fallisce comunque. Prendete l'abitudine: `.isdecimal()` per i numeri.

13.5 Controllo 2 bis: .replace() e il terzo argomento

```
# 1. Normalizzazione all'italiana: la virgola dei decimali diventa punto.
# Non è un vezzo: float("12,90") si ferma con l'eccezione ValueError.
testo = "12,90".strip().replace(",",".")

# 2. Il terzo argomento è il tetto delle sostituzioni: toglì UN punto solo.
# Se quello che resta è fatto di sole cifre, il numero era ben formato.
cifre = testo.replace(".", "", 1)

print(testo)                # 12.90
print(cifre.isdecimal())    # True

# TRABOCCHETTO: senza il tetto di 1 anche "12.9.0" passerebbe il controllo.
# Qui dopo averne tolto uno resta un punto, .isdecimal() dice False, e la
# riga viene respinta come merita.

print("12.9.0".replace(".", "", 1).isdecimal()) # False
```

Qui non c'è un metodo pronto: il trucco è togliere un solo punto e guardare cosa resta. Attenzione a che cosa NON copre: cifre è una stringa e resta tale. Il valore si converte con float(testo), dopo, e solo nel ramo buono.

13.6 Controllo 3 — intervallo: il valore plausibile

```
QUANTITA_MASSIMA = 100

testo_quantita = "500"

# 1. Prima la forma. Questo ramo è la guardia che protegge i due successivi:
#     è l'unico posto in cui si scartano le stringhe non numeriche.
if not testo_quantita.isdecimal():

    print("[ERRORE] La quantità deve essere un numero intero.")

# 2. Poi l'intervallo, e solo adesso int() si può chiamare senza rischi.
#     Zero pezzi è un numero valido e un ordine assurdo: tipo giusto,
#     valore no, e nessun metodo di stringa se ne accorge.
elif int(testo_quantita) == 0:

    print("[ERRORE] La quantità deve essere almeno 1 pezzo.")

elif int(testo_quantita) > QUANTITA_MASSIMA:

    print(f"[ERRORE] Massimo {QUANTITA_MASSIMA} pezzi per scontrino.")

else:

    print(f"[OK] {testo_quantita} pezzi registrati")

# TRABOCCHETTO: invertite il primo elif con l'if e il programma torna a
# fermarsi con ValueError. L'ordine dei rami non è stile: è la condizione
# perché int() venga eseguito solo su cifre.
```

13.7 Controllo 4 — valori ammessi: confronti uniti da or

Dopo `zona = input(...).strip().upper()`, la condizione è:

```
if zona == "NORD" or zona == "CENTRO" or zona == "SUD" or zona == "ISOLE":
```

con l'else che stampa la zona rifiutata e l'elenco di quelle servite.

Quattro confronti in fila sono verbosi. Esiste una forma compatta che usa una lista, e arriva quando parleremo di strutture dati.

Fino ad allora si scrive per esteso, con `or` fra un confronto e l'altro, ricordandosi del trabocchetto del paragrafo 8.4, che è esattamente in agguato qui.

13.8 Il messaggio d'errore: cosa non va, dove, cosa fare

Messaggio scadente	Messaggio utile
Errore	[ERRORE] La quantità deve essere un numero intero.
Input non valido	[ERRORE] L'ISEE va scritto senza punti: 14500, non 14.500.
ValueError	[ERRORE] Il prezzo deve essere un numero, per esempio 12,90.
Campo obbligatorio	[ERRORE] Il protocollo è obbligatorio: è in alto a destra sul modulo.

Tre regole valide anche fuori da Python: niente parole di Python (l'utente non sa cosa sia un ValueError e non deve saperlo); un esempio vale dieci spiegazioni; dare sempre la via d'uscita. Lo sportello che dice subito "manca la firma a pagina 2" è più utile di quello che accetta tutto e richiama dopo tre settimane.

13.9 Lo schema completo: lettura, errore, catena di elif

```
ALIQUOTA_IVA = 0.22

QUANTITA_MASSIMA = 100

# 1. Lettura e normalizzazione, tutte insieme e prima di ogni controllo.
#     Dopo queste tre righe in memoria ci sono solo stringhe: nessun numero.
testo_prezzo = input("Prezzo unitario in euro: ").strip().replace(",", ".")
testo_quantita = input("Quantità: ").strip()
cifre_prezzo = testo_prezzo.replace(".", "", 1)

# 2. L'accumulatore degli errori parte VUOTO, cioè falsy (paragrafo 11.2).
#     Finché resta vuoto, non è stato trovato niente di storto.
errore = ""

# 3. Una catena di elif, non if separati: al primo problema trovato ci si
#     ferma. Segnarne quattro insieme confonde chi sta alla cassa.
if not cifre_prezzo.isdecimal():
    errore = "Il prezzo deve essere un numero, per esempio 12,90."
elif not testo_quantita.isdecimal():
    errore = "La quantità deve essere un numero intero, senza decimali."
elif int(testo_quantita) == 0:
    errore = "La quantità deve essere almeno 1 pezzo."
elif int(testo_quantita) > QUANTITA_MASSIMA:
    errore = f"Massimo {QUANTITA_MASSIMA} pezzi per scontrino."
```

13.9 Lo schema completo: il ramo buono e le conversioni

4. Un solo punto di stampa: il messaggio è scritto una volta, non quattro.

if errore:

```
    print(f"[ERRORE] {errore}")
```

```
    print("Nessuno scontrino emesso. Ripetere l'operazione.")
```

else:

```
    # TRABOCCHETTO: le conversioni stanno qui, nel ramo buono, e mai in
```

```
    #   cima. Spostate float() o int() sopra la catena e il programma torna
```

```
    #   a fermarsi prima ancora di poter stampare il vostro messaggio.
```

```
    prezzo = float(testo_prezzo)
```

```
    quantita = int(testo_quantita)
```

```
    totale = prezzo * quantita * (1 + ALIQUOTA_IVA)
```

```
    print(f"Totale: {totale:.2f} euro")
```

Le tre cose da rubare a questo schema sono numerate nei commenti: la variabile errore che parte vuota, la catena che si ferma al primo problema, il punto di stampa unico. La quarta è nel TRABOCCHETTO, ed è quella che in aula si sbaglia.

13.10 I due limiti dichiarati: niente ciclo, niente cattura

Il programma finisce comunque.

se il dato è sbagliato si stampa il messaggio e si chiude
per richiedere il dato finché non è buono serve un ciclo
i cicli arrivano più avanti, ed è l'esercizio 08.4

Esiste un altro modo di affrontare gli errori.

uno strumento apposito raccoglie l'errore DOPO che è successo,
invece di prevenirlo: si chiama try/except
non è materia di queste undici giornate, e in nessun esercizio compare

Qui ci fermiamo alla validazione preventiva, che su un dato digitato
da una persona resta la difesa giusta. Sapere che l'altra strada esiste
vi serve per non credere che `if` e `.isdecimal()` siano tutto quello che c'è.

14. DAL FOGLIO AL CODICE

Alberi di decisione e punteggi

14.1 Albero di decisione e classificatore a punteggio

	Albero di decisione	Punteggio e soglia
Quando si usa	regole nette, poche	molti indizi deboli
Come si scrive	if / elif / else	if separati che sommano
Cosa produce	una categoria	un numero, poi una categoria
Come si corregge	si sposta una regola	si sposta un peso o la soglia
Esempio di oggi	triage 04.3	scrematura 04.7

Il secondo serve quando nessun indizio da solo decide, e sta sotto a parecchi sistemi che decidono al posto di qualcuno: il filtro della posta indesiderata, il controllo antifrode della banca, la prima scrematura delle candidature. Lo vedrete tornare in ogni giornata da qui alla fine del blocco.

14.2 Dal diagramma alla catena: due rami fusi con or

```
cosciente?  --no--> ROSSO
  |
  sì
  v
dolore >= 8 ?  --sì--> ROSSO
  |
  no
  v
dolore >= 5 ?  --sì--> GIALLO
  |
  no
  v
VERDE
```

I due rami "ROSSO" si fondono in una condizione sola con or, ed esce la catena del capitolo 5 con not cosciente or dolore >= DOLORE_ROSSO come primo ramo. Disegnare prima non è un passaggio scolastico: è il momento in cui ci si accorge che manca un caso, e costa trenta secondi invece di due ore.

14.3 Indizi deboli e pesi: il tariffario delle parole spia

Parola spia	Peso	Perché da sola non basta
gratis	3	lo scrive anche una promozione legittima
vinci	3	e lo contiene anche la parola provincia
clicca subito	4	l'indizio più forte, insieme a credenziali
offerta imperdibile	2	commerciale, non per forza spam
urgente	1	lo scrive anche un cliente arrabbiato
credenziali	4	la parola delle truffe vere
rimborso	2	lo scrive chi ha diritto a un rimborso

Nessun messaggio si riconosce da un solo indizio: quello che li distingue è l'accumulo. Soglia: 5 punti, da 5 in su il messaggio è classificato come spam. Da qui alla fine del blocco questo tariffario non cambia: lo ritrovate identico ogni volta che il filtro torna.

14.4 L'operatore di appartenenza in: sequenza, non parola

```
messaggio = "Offerta imperdibile: clicca subito e ricevi un buono gratis"

# Normalizzazione: .lower() restituisce una copia tutta minuscola. Il dato
# originale resta intatto, perché le stringhe sono immutabili.
testo = messaggio.lower()

# 1. La sequenza c'è: True. L'operando di sinistra è quello cercato, quello
# di destra è il testo in cui cercare. Invertirli cambia la domanda.
print("gratis" in testo)                # True

# 2. La sequenza non c'è: False. Nessun errore, solo un valore di verità.
print("credenziali" in testo)           # False

# TRABOCCHETTO: i due casi da cui nasce la regola di questa sezione.

# 3. False: qui il testo NON è normalizzato e "GRATIS" è maiuscolo.
# 4. True: dentro "provincia" c'è la sequenza "vinci", e un messaggio del
# Comune si becca tre punti per niente.
print("gratis" in "Servizio GRATIS per tutti")    # False
print("vinci" in "il nuovo bando della provincia") # True
```

14.5 Il classificatore completo: pesi, soglia, accumulatore

```
# Filtro spam a punteggio - NovaStore, versione del Giorno 04

# Il tariffario del paragrafo 14.3, un peso per costante: ritoccare un peso
# vuol dire cambiare un numero qui in cima, non cercarlo dentro la logica.

PESO_GRATIS = 3

PESO_VINCI = 3

PESO_CLICCA_SUBITO = 4

PESO_OFFERTA_IMPERDIBILE = 2

PESO_URGENTE = 1

PESO_CREDENZIALI = 4

PESO_RIMBORSO = 2

# La manopola del paragrafo 14.6: da 5 punti in su il messaggio è spam.

SOGLIA_SPAM = 5

def main():

    messaggio = "Offerta imperdibile: clicca subito e ricevi un buono gratis"

    # 1. Normalizzazione prima della ricerca: `in` distingue le maiuscole.

    testo = messaggio.lower()

    # 2. L'accumulatore parte da zero e cresce a ogni indizio trovato.

    punteggio = 0
```

14.5 Il classificatore completo: sette if separati

```
# 3. Sette if SEPARATI, non una catena di elif: le parole spia possono
#   comparire tutte insieme, e ognuna aggiunge i suoi punti.

if "gratis" in testo:
    punteggio = punteggio + PESO_GRATIS

if "vinci" in testo:
    punteggio = punteggio + PESO_VINCI

if "clicca subito" in testo:
    punteggio = punteggio + PESO_CLICCA_SUBITO

if "offerta imperdibile" in testo:
    punteggio = punteggio + PESO_OFFERTA_IMPERDIBILE

if "urgente" in testo:
    punteggio = punteggio + PESO_URGENTE

if "credenziali" in testo:
    punteggio = punteggio + PESO_CREDENZIALI

if "rimborso" in testo:
    punteggio = punteggio + PESO_RIMBORSO


# TRABOCCHETTO: mettete elif al posto del secondo if e la catena si
#   ferma al primo indizio trovato: 3 punti invece di 9.
print(f"Punteggio: {punteggio} (soglia {SOGLIA_SPAM})")


# 4. Il punteggio è un numero, la categoria nasce dalla soglia.
if punteggio >= SOGLIA_SPAM:
    print("[!] Messaggio classificato come SPAM")
else:
    print("[OK] Messaggio consegnato in posta in arrivo")
```

Punteggio: 9 (soglia 5), poi [!] Messaggio classificato come SPAM. Nove punti: due per offerta imperdibile, quattro per clicca subito, tre per gratis.

14.5 Perché sette if separati e non una catena di elif

Le parole spia possono comparire tutte insieme,
e ognuna deve poter aggiungere i suoi punti.

Con elif il messaggio di prima si sarebbe fermato al primo ramo vero:
3 punti invece di 9, e sarebbe finito in posta in arrivo.

È la domanda del capitolo 5: i casi si escludono a vicenda?
Qui no, convivono. Quindi if separati.

L'accumulatore: $\text{punteggio} = 0$, poi $\text{punteggio} = \text{punteggio} + \text{PESO_...}$
è lo stesso schema del totale di uno scontrino

Sommo punti, confronto con una soglia, decido: da oggi alla fine del blocco
questo meccanismo non cambia.

14.6 Spostare la soglia: cosa si guadagna e cosa si perde

Messaggio	Punti	Con soglia 5	Con soglia 3
"Le confermo l'appuntamento di giovedì"	0	consegnato	consegnato
"Urgente: pratica di rimborso in scadenza"	3	consegnato	SPAM
"Vinci un buono gratis, clicca subito"	10	spam	spam

La riga di mezzo è quella interessante: con la soglia a 3 un cliente che chiede un rimborso urgente non riceve risposta. Abbassare la soglia significa prendere più spam e buttare via più posta buona. Non esiste la soglia giusta in assoluto: esiste quella giusta per il costo dell'errore che vi fa più male.

14.7 Il filo conduttore: dove torna questo classificatore

Dove	Esercizio	Cosa diventa
Giorno 05	05.2	lo stesso filtro, con il messaggio digitato dall'utente
Giorno 07	07.1	il classificatore con la soglia spostabile e le sue misure
Giorno 08	08.7	lo stesso filtro su una sequenza di messaggi, con un ciclo
Giorno 09	09.5	la logica racchiusa in una funzione riutilizzabile
Giorno 11	11.5	applicato a un file di trenta ticket, con un report su disco

Non è un esercizio usa e getta: è il filo conduttore del corso. Quello che oggi fate su un messaggio, nell'ultima giornata lo farete su un file intero. Il meccanismo non cambia mai: è quello che avete appena scritto.

15. ERRORI LOGICI

Il programma gira e sbaglia

15.1 Sintassi, esecuzione, logica: chi ve li segnala

	Errore di sintassi	Errore di esecuzione	Errore logico
Quando si manifesta	prima di partire	mentre gira	mai
Chi ve lo dice	Python	Python	nessuno
Cosa vedete	non parte	il traceback	un numero plausibile
Quanto costa	dieci minuti	mezz'ora	un pomeriggio, o peggio

Da oggi in poi la maggior parte dei vostri problemi sarà della terza colonna. Le condizioni sono la porta d'ingresso degli errori logici, perché una condizione sbagliata produce comunque una risposta. Il programma che dice "sconto 5%" invece di "sconto 15%" non è rotto: è peggio che rotto, perché nessuno lo controlla.

15.2 La tabella dei casi limite: i cinque punti

Dove provare	Perché
sotto la soglia	il ramo basso funziona?
esattamente sulla soglia	$> o \geq$? È qui che si sbaglia
appena sopra la soglia	il ramo alto scatta?
il valore assurdo	zero, negativo, enorme
il campo vuoto o non numerico	il programma sopravvive?

Non esiste un modo di guardare un if e vedere se è giusto. Esiste un modo di provarlo dove fa male, e sono sempre questi cinque posti. La colonna "atteso" si compila a mano PRIMA di eseguire; poi si lancia il programma e si riempie la colonna "ottenuto". Scritta così, la tabella è anche la prova da mostrare a chi vi ha commissionato il programma.

15.3 La tabella compilata sulla catena del paragrafo 6.1

Spesa	Atteso	Ottenuto	Esito
0.00	0%	0%	[OK]
49.99	0%	0%	[OK]
50.00	5%	5%	[OK]
99.99	5%	5%	[OK]
100.00	10%	5%	[!]
199.99	10%	5%	[!]
200.00	15%	5%	[!]
1000.00	15%	5%	[!]

Quattro righe su otto sbagliate e quattro giuste: ecco perché il difetto è arrivato in produzione. Chi ha provato il programma ha fatto due prove con spese piccole, ha visto due risposte giuste e ha chiuso il file. Se non l'avete provato sul bordo, non l'avete provato.

15.4 Condizione sempre falsa: un and dove serviva un or

```
zona = "NORD"

# TRABOCCHETTO: i due operandi confrontano la STESSA variabile con due
# valori diversi. Perché l'and sia vero, zona dovrebbe valere "NORD" e
# "CENTRO" nello stesso istante: non succede per nessun dato, quindi il
# primo ramo è irraggiungibile e l'else scatta sempre.
if zona == "NORD" and zona == "CENTRO":
    print("Tariffa continentale")
else:
    # Qui finisce anche la spedizione al NORD, che la tariffa ce l'avrebbe.
    print("Tariffa fuori rete")

# Tariffa fuori rete
```

È il gemello della condizione sempre vera del paragrafo 8.4. Il rimedio è quello del paragrafo 8.5: leggere la condizione ad alta voce. Quando gli operandi di un and confrontano la stessa variabile con valori diversi, l'operatore giusto è or.

15.4 La riga fuori dal blocco: il prelievo negato

```
SALDO_INIZIALE = 120.00

saldo = SALDO_INIZIALE

prelievo = 300.00

# La guardia c'è ed è giusta: 300 non è minore o uguale a 120, quindi la
# riga rientrata non viene eseguita e nessuna autorizzazione è stampata.
if prelievo <= saldo:

    print("Prelievo autorizzato")

# TRABOCCHETTO: questa riga è a colonna 0, quindi è FUORI dal blocco
# dell'if. Si esegue sempre, anche quando il prelievo è stato negato.
# Bastano quattro spazi per rimetterla dentro.

saldo = saldo - prelievo

print(f"Saldo: {saldo:.2f} euro")

# Saldo: -180.00 euro
```

Trento euro spariti da un conto per un'operazione mai avvenuta, senza nessun messaggio d'errore. È il blocco 3 dell'esercizio casa_04.4: se stasera fate un esercizio solo, fate quello.

15.5 Il collaudo incrociato: chi scrive non è chi collauda

Chi ha scritto un programma è la persona meno adatta a collaudarlo:

prova i casi a cui ha pensato mentre lo scriveva,
che sono esattamente quelli che funzionano.

Nel laboratorio del pomeriggio, sull'esercizio 04.6, si lavora in coppia

con i ruoli separati:

uno scrive il codice

l'altro compila la tabella dei casi limite e prova a farlo cadere

poi ci si scambia

Non è un gioco: è la prima forma di collaudo del corso,

ed è come funziona nei gruppi di lavoro veri.

16. ERRORI TIPICI DELLA GIORNATA

Quelli che si vedono e quelli che non si vedono

16.1 I quattro errori segnalati da Python

```
if voto = 30:  
    ^^^^^^^^
```

SyntaxError: invalid syntax. Maybe you meant '==' or ':=' instead of '='?

```
if totale >= SOGLIA_SPEDIZIONE_GRATUITA  
                                     ^
```

SyntaxError: expected ':'

```
if voto > 18 and < 30:  
                ^
```

SyntaxError: invalid syntax

```
print("Grazie")  
      ^
```

TabError: inconsistent use of tabs and spaces in indentation

Il programma non parte: sono i meno costosi della giornata, perché Python ve li dice in faccia. In Excel = è il confronto, ed è per questo che il primo lo fanno tutti. L'indentazione mista nasce quasi sempre da un copia-incolla dalla chat.

16.2 else allineato all'if sbagliato: il rientro decide

```
MAGGIORE_ETA = 18
SOGLIA_ISEE = 8000

eta = 25
isee = 12000

# 1. if esterno, colonna 0: la domanda è sull'età.
if eta >= MAGGIORE_ETA:
    # 2. if interno, colonna 4: la domanda è sull'ISEE, e viene posta
    # solo quando la prima ha risposto sì.
    if isee <= SOGLIA_ISEE:
        print("Agevolazione piena")
# TRABOCCHETTO: questo else sta a colonna 0, quindi appartiene all'if
# sull'età e si legge "se è minorenni". Con eta = 25 non scatta, e il
# caso "maggiorenne con ISEE alto" resta senza nessun ramo.
else:
    print("Nessuna agevolazione")
```

16.2 else allineato all'if sbagliato: la versione corretta

```
MAGGIORE_ETA = 18
SOGLIA_ISEE = 8000

eta = 25
isee = 12000

if eta >= MAGGIORE_ETA:
    if isee <= SOGLIA_ISEE:
        print("Agevolazione piena")

        # Colonna 4: appartiene all'if sull'ISEE, si legge "ISEE sopra la
        # soglia". È il ramo che mancava.
    else:
        print("Nessuna agevolazione")

# Colonna 0: appartiene all'if sull'età, si legge "è minorenni".
else:
    print("Pratica a carico di un genitore")
```

Due else, due colonne, due domande: il rientro è l'unica cosa che dice a quale domanda ogni ramo sta rispondendo. Quando un else non fa quello che vi aspettate, contate gli spazi: dicono di chi è.

```
# Nessuna agevolazione
```

16.3 I tre errori senza eccezione: or nudo, == fra tipi

`if nome == "mario" or "Mario"`

sempre vero, il paragrafo 8.4 spiega perché

è il trabocchetto più insidioso di oggi ed è il primo blocco di casa_04.4

Confrontare un testo con un numero

con `>=` si ottiene `TypeError: '>=' not supported between instances`

of `'str'` and `'int'`, e almeno ve ne accorgete

con `==` non c'è nessun errore: solo una condizione eternamente falsa

Dimenticare `.strip()` e `.lower()`

l'utente risponde sì, il controllo capisce no, e nessuno protesta

è il caso della prossima slide

16.3 Dimenticare .strip() e .lower(): il testo sporco

```
# Il dato com'è arrivato: maiuscola iniziale e uno spazio finale che a
# schermo non si vede. Tipo str, lungo 3 caratteri.
risposta = "Si "

# 1. Confronto sul dato grezzo: "Si " e "si" differiscono in due punti,
#   la prima lettera e lo spazio in coda. Risultato False.
if risposta == "si":
    print("Confermato")
else:
    print("Non confermato")

# 2. Stesso confronto sul dato normalizzato. .strip() toglie il bianco ai
#   bordi, .lower() porta tutto in minuscolo: i due metodi restituiscono
#   stringhe nuove e non toccano risposta, che resta "Si ".
if risposta.strip().lower() == "si":
    print("Confermato davvero")

# TRABOCCHETTO: qui i rami sono due if separati, non un if/else. Per questo
#   escono due righe: la prima decisione ha già stampato il suo verdetto.
```

L'utente ha risposto sì, il primo controllo ha capito no, e nessuno ha protestato.

```
# Non confermato

# Confermato davvero
```

16.4 Pronto soccorso: i messaggi di Python

Messaggio	Cosa è successo	Prima cosa da guardare
SyntaxError: expected ':'	mancono i due punti	fine della riga dell'if
SyntaxError: ... Maybe you meant '=='	= in una condizione	l'operatore di confronto
SyntaxError: invalid syntax su else:	else disallineato	il margine sinistro dell'else
IndentationError: expected an indented block	blocco vuoto dopo i due punti	la riga sotto l'if
IndentationError: unexpected indent	spazi di troppo	il margine sinistro della riga
TabError	tabulazioni e spazi mescolati	tutto il bianco a inizio riga
TypeError: '>=' not supported...	confronto fra testo e numero	manca int() o float()
ValueError: invalid literal for int()	conversione di un testo non numerico	validare con .isdecimal()
NameError: name 'x' is not defined	variabile creata solo dentro un ramo	manca l'else finale

Colonna di sinistra quello che vedete a schermo, colonna di destra il punto del codice da guardare per primo. Da tenere aperta durante i laboratori.

16.4 Pronto soccorso: i sintomi senza messaggio

Sintomo	Cosa è successo	Prima cosa da guardare
Esce sempre lo stesso ramo	soglie in ordine sbagliato	l'ordine degli elif
L'else non scatta mai	condizione sempre vera	un or con un valore da solo
L'else scatta sempre	condizione sempre falsa	un and dove serviva un or
Succede qualcosa che non doveva	riga fuori dal blocco	i quattro spazi

Qui Python non dice niente, e la colonna di sinistra è l'unica cosa che avete. Sono i quattro sintomi del capitolo 15: quando ne riconoscete uno, la strada non è rileggere il codice, è costruire la tabella dei casi limite.

16.5 Il metodo di diagnosi, in sei passi

1. Non toccare niente. Leggete.
2. Ultima riga del messaggio: che errore è.
3. Riga con File: dove.
4. Aprite quella riga, e guardate anche quella sopra.
5. Una modifica alla volta, poi rilanciate.
6. Nuovo da oggi, per gli errori che non danno errore:
se non c'è nessun messaggio ma il risultato è sbagliato,
non cercate nel codice, costruite la tabella dei casi limite.
Il programma vi dirà da solo dove sbaglia.

Riepilogo del giorno (1/2)

if = condizione + due punti + blocco rientrato. Il rientro È il blocco.

Quattro spazi, mai la tabulazione. Un rientro sbagliato non dà errore:

dà un programma diverso.

if/else: due strade, una sola percorsa, sempre una; else allineato con il suo if.

Nei rami si assegna, alla fine si stampa.

if/elif/else: il primo ramo vero vince, gli altri non vengono nemmeno valutati.

elif se i casi si escludono; if separati se convivono.

Con $\geq$ dalla soglia più alta in giù, con $\leq$ dal basso in alto; mai mescolare.

"Da 50" include il 50, "oltre 50" no: il valore esatto della soglia

è il primo caso da provare.

Sulle stringhe contano maiuscole e spazi: `.strip().lower()` prima di confrontare.

Mai `==` fra due numeri con la virgola: si usa una tolleranza.

Riepilogo del giorno (2/2)

Precedenza not → and → or; quando compaiono insieme, parentesi sempre.

nome == "a" or "b" è sempre vero: "b" da solo è un valore, non un confronto.

Valutazione pigra: il controllo che protegge gli altri va per primo.

Confronti concatenati: $18 \leq \text{eta} < 65$, e gli operatori nella stessa direzione.

Falsi: False, 0, 0.0, "". Veri: " " e "0".

if campo: sui testi, confronto esplicito sui numeri.

Annidamento: massimo due livelli; con una riga sola dentro, si appiattisce con and.

Validare in quattro passi: presenza, forma, intervallo, valori ammessi.

Convertire dopo, e solo nel ramo buono.

in cerca una sequenza, non una parola, e distingue le maiuscole.

Punteggio e soglia: if separati che sommano, poi il confronto con la soglia.

Gli errori logici si trovano solo con la tabella dei casi limite, sul bordo.

Grazie.

shaping the skills of tomorrow

